Design and Implementation of a Custom Multiply Accumulate Instruction in an Open-Source RISC-V Processor Core

S. Hemajothi
Professor
Electronics and Communication Engineering,
Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Chennai, India
hemajothi.s@velhightech.com

S. Jalaja
Assistant Professor
Electronics and Communication Engineering
Vel Tech High Tech Dr. Rangarajan Dr Sakunthala Engineering College
Chennai, India
jalaja@velhightech.com

Kishore M
Electronics and Communication Engineering
Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College
Chennai, India
vh12519_ece22@velhightech.com

Gowtham Kumar S
Electronics and Communication Engineering, Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Chennai, India
vh12495_ece22@velhightech.com

Gowtham Kumar S
Electronics and Communication Engineering, Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College, Chennai, India
vh12494_ece22@velhightech.com

2026 9th International Conference on Circuit, Power & Computing Technologies (ICCPCT) | 979-8-3315-5198-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICCPCT70290.2026.11654754

Abstract –
With the rise in popularity of applications requiring high-performance arithmetic acceleration, there's an increasing need for architectural enhancements to low-power embedded and signal processor cores. This paper discusses the design and hardware implementation of an application-specific Multiply-Accumulate (MAC) instruction developed for the open-source PicoRV32 RISC-V processor using the Pico Co-Processor Interface (PCPI). A new opcode-based decode mechanism will enable this process, where a specific field will be checked in the instruction to activate the MAC operation, without having to change the underlying RISC-V ISA. Register-to-register execution will be possible in this hardware MAC-based architecture because it will allow for two-operand forwarding from the processor register file to the hardware MAC unit and back through the PCPI for write-back.

The design has been implemented in Verilog and verified through functional simulation and FPGA synthesis by using Vivado. Synthesis results indicate that the MAC-enabled processor utilizes an area-efficient design, where 1,338 LUTs, 953 flip-flops, 2 DSP slices, and very little BRAM were used. Performance metrics show that significant reductions in execution cycles were achieved from the MAC-enabled processor versus a baseline RISC-V processor and therefore validates the efficacy of the proposed custom instruction for compute-intensive workloads

Keywords- RISC-V processor, Custom instruction extension, Multiply-Accumulate unit, PicoRV32, Processor architecture, RTL design, Performance optimization

I. INTRODUCTION

The demands of modern software applications, including Machine Learning (ML), Signal Processing (SP), Machine Vision (MV), and Edge Computing (EC), have risen substantially. They also require faster and more efficient execution of multiply and accumulate (MAC) operations, which are at the heart of many data-intensive algorithms. Since efficient implementation of MACs will have a direct effect on both overall system performance and energy usage [6], [13], there has been a noticeable increase in the use of open-source Instruction Set Architectures (ISAs) such as RISC-V. RISC-V has gained popularity due to its ability to easily accommodate custom architectural extensions through the use of application specific instructions (ASICs). RISC-V allows for more efficient processor design than traditional ISAs, while still applying the same ISA structure to a new processor design. Of all currently available open-source RISC-V processor cores, the PicoRV32 processor architecture has become one of the most commonly used embedded microcontrollers for resource constrained environments because of its low power consumption, modular architecture, and small physical footprint [12].

Generally speaking, in baseline operating (or basic) configuration of RISC-V processor, MAC instruction-type operation are executed via two distinct sets of instruction(s): multiply and add (or sum). Therefore, operating via multiple instructions increases instruction-count and overall-latency, particularly for workloads that involve repetitive MAC calculations, resulting in creating less efficient–throughput for improved performance [6], [7].

979-8-3315-5198-8/26/$31.00 ©2026 IEEE

The custom extensions of the open-source RISC-V architecture have been designed in accordance with the principles of RISC-V's extension development (i.e., having extended instructions encoded in machine language), which have allowed us to demonstrate how to design and implement a custom multiply-accumulate (MAC) extension to the PicoRV32 using the Pico Co-Processor Interface (PCPI). The dedicated co-processor MAC responds to a unique opcode and has been designed to communicate with the processor through multiple PCPI handshake signals, which has allowed for a modular interface between the two devices; this prevents any changes being made to the internal data bus of the processor. The MAC was described in Verilog RTL and successfully validated through functional testing, cycle-accurate simulation and synthesis of the actual hardware. The results obtained from the testing indicate that the MAC instruction can be executed within a single processor cycle and only with very minimal additional hardware overheads, demonstrating the potential benefits of custom extensions to improve the operational performance of a small open-source RISC-V processor [15].

I. PROPOSED ARCHITECTURE

The architecture proposed integrates a custom Multiply-Accumulate (MAC) instruction into a lightweight opensource RISC-V processor core, PicoRV32, by means of the Pico Co-Processor Interface (PCPI) [2]. As illustrated in Fig. 1, the overall system architecture consists of instruction memory, PicoRV32 core (instruction fetch/decode), control unit, arithmetic logic unit (ALU), PCPI-based custom MAC co-processor, and write back path. The modular integration of the custom MAC co-processor allows for efficient execution of MAC operations without having to alter the internal datapath of the processor core [6].

Within the proposed design, the PicoRV32 core performs all instruction fetch, decode and control signal generation functions [7]. Instruction data is obtained from a block of on-chip instruction memory and decoded within the core. Upon receipt of an instruction, the control unit determines if the current instruction is a standard RISC-V instruction or a custom MAC (multiply-accumulate) instruction. For all other types of arithmetic or logical operations, these instructions will be executed by the ALU using the standard datapath. If an instruction is identified as a custom MAC opcode, the instruction will be offloaded (transferred) to the PCPI-based MAC co-processor for execution at an increased rate than would normally occur through the RISC-V data path [10].

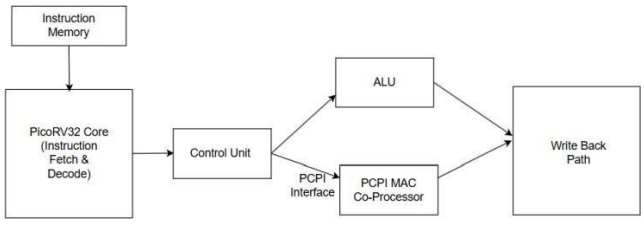


Fig 1. PICORV32 ARCHITECTURE

A connection between the PicoRV32 CPU and an external piece of custom hardware (a co-processor) has been established via the PCPI interface. The PCPI interface utilizes a total of three handshake signals and several result signals to control the completion of the instruction being executed by the external unit and to maintain the synchronization between the two devices (the CPU executing the custom instruction and the co-processor that will return results to the CPU) [9]. When a custom instruction is created by the CPU, it sends two source operands (rs1 and rs2) to the PCPI MAC co-processor for further processing. Upon receipt of the custom opcode, the PCPI MAC co-processor will decode that custom opcode and activate a dedicated (this MAC processing unit is exclusively for this custom operation) MAC processing unit to perform a single execute signed multiply accumulate operation in one instruction stream. Therefore, if you were to perform the same function on an external co-processor, you would have to execute a number of sequential instructions using the base RISC-V ISA in order to complete the operation [11].

In order to prevent conflicts between elements in the pipeline, a dedicated RTL Module was designed specifically for the MAC Device to be able to act independently of the AALU operation. The MAC Device will send results back to the processor through the PCPI result channel and the results will then be written back to the register file as part of a normal writing back to register file process. Because this type of co-processor integration is used, the design of the PicoRV32 architecture allows for simple, compact designs while applying physical resources (hardware accelerators) to MAC-related workloads [23]. The modular nature of the PicoRV32 design also enables scalability with a lower hardware footprint compared to other solutions currently available, and supports RISC-V compliant execution; therefore making the PicoRV32 an excellent solution for any application where engineered arithmetic acceleration is required in an embedded or edge-based environment.

III. CUSTOM MAC INSTRUCTION DESIGN

Utilization of an in-house-designed multiply-accumulate instruction will provide hardware acceleration for MAC operations in the pico co-processor interface of the PicoRV32 processor [8]. A MAC operation executed via traditional RISC-V requires two separate instructions - multiply and add. As a result, the number of instructions needed increases, and the latency of getting the work done gets longer, resulting in less efficient use of workloads such as signal processing and machine learning that utilize this operation.

Mathematically, the MAC operation being implemented in this project can be represented as follows:

$$MAC_{out} = Acc + (a\ x\ b)$$

Input operands a and b and accumulator in Acc. This representation matches RTL construction of MAC unit, single execution path updating the accumulator with the product of operands. Custom instruction is identified by custom opcode in custom instruction space of RISC-V ISA.

Upon decoding, when the custom MAC opcode is detected, the PCPI interface starts operating, with operands from registers rs1 and rs2 forwarded to the PCPI MAC processor. Execution is controlled by handshake signals (pcpi_valid and pcpi_ready) and computed results returned through pcpi_rd and written back to register file.

The latency improvement in this project can be represented as follows:

$$Latency_{custom} < Latency_{mul} + Latency_{add}$$

The cycle-level simulation proves that the single-cycle execution of the custom MAC instruction, which reduces the instruction overhead and also improves the computational efficiency with minimal modification [9].

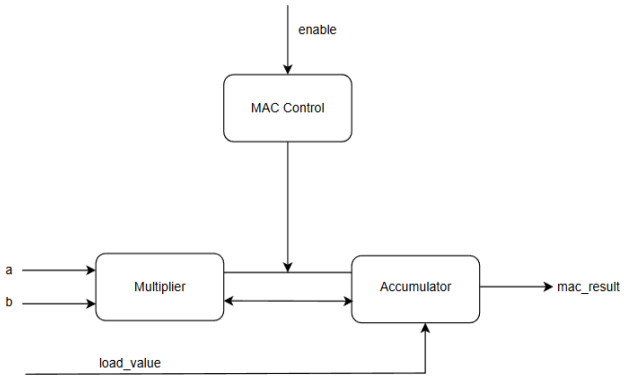


Fig 2. Mac Block

1 . MAC Instruction Overview

A R-type instruction encoding of the RISC-V ISA is used for the proposed MAC instruction and involves 2 source registers and one destination register [1] [4]. On the basic processor, multiplies and additions are distinct instructions, one for the multiply then another to add. Instead, the new instruction performs two operations in a single atomic operation, reducing the number of instructions [3].

Functionality of the MAC instruction is:

rd←rd+(rs1×rs2)

where the source-register operands are rs1 and rs2 while rd is a destination (and accumulator) register [6]. This architecture does not require a dedicated accumulator register, and can be integrated into the present general-purpose register file design.

To distinguish the Custom MAC instruction from other ALU operations, a global unique identifier along with a unique opcode/function will be defined. During the decode step, the global unique identifier will be used to differentiate between the ALU and the Custom MAC execution path control signals and activate the Custom MAC execution path [15].

Table: Custom MAC Instruction Encoding Format

Field	Bits	Description
opcode	[6:0]	Custom MAC opcode
rd	[11:7]	Destination / Accumulator register
funct3	[14:12]	MAC operation identifier
rs1	[19:15]	Source Register 1
rs2	[24:20]	Source Register 2
funct7	[31:25]	MAC function code

2 . MAC Unit Architecture

The inner shape of the proposed MAC unit is shown in Fig. 2. The MAC block consists of a devoted multiplier, accumulator, and control common sense. The multiplier generates the made of its input operands, and the accumulator provides the product to a previously saved fee. Enable, load, and write-again operations of the MAC unit are managed by way of the control good judgment the use of decoded signals from commands [6], [17].

The MAC unit is applied along the arithmetic good judgment unit (ALU) inside the processor's execute level. The ALU and MAC devices function as parallel execution devices, managed by the manipulate unit in response to the decoded preparation type. This parallel configuration allows each MAC and preferred mathematics operations to execute with none practical conflicts [14], [15].

IV. DATAPATH AND CONTROL LOGIC

The PicoRV32 architecture has been improved by adding a dedicated multiply-accumulate (MAC) capability to its current ALU in order to allow the architecture to easily support the proposed MAC instruction (which will only exist in the decode stage of the PicoRV32 when the instruction has a MAC operation) [1], [4]. The RISC-V standard instruction set will continue to function exactly as it does without any additional modification. The MAC capability will enable some of the MAC instructions to use the new MAC unit in conjunction with other instructions that do not contain MAC functionality [15].

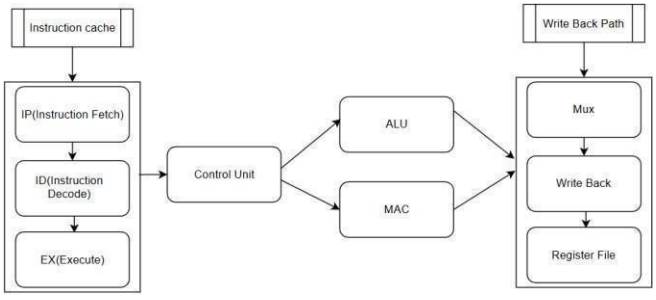


Fig.3 Mac in the PICORV32

During execution of the PicoRV32, both the MAC unit and the ALU will be used at the same time. The two units will have access to the operands stored in the processor's register file. The decode circuitry will indicate whether the instruction will be executed within the ALU or the MAC unit, allowing for complete compatibility with the baseline instructions [14].

The Custom MAC instruction's decode path should be separate from the standard decode path for the other instructions (ALU based). When the Control Logic detects that the Instruction is a Custom MAC instruction, it will activate the appropriate MAC signals (Enable MAC) and route the Operands to the MAC Unit
- that is, both Register File Operands will be routed to the MAC unit - and deactivate the ALU for the execution of that Instruction

V. RTL IMPLEMENTATION

The Register Transfer Level (RTL) representation in Verilog for the custom MAC instruction system has been established. The custom MAC design uses three components to create an overall modular design that can scale, reuse, and work with the fundamental architecture of the PicoRV32 core. Those components are: the PicoRV32 processor core; the interfacing with the MAC through the Processor Co-Processor Interface (PCPI) system; and the MAC calculation logic [21].

The PicoRV32 core has been modified so that it can now support PCPI, allowing for an external co-processor to execute custom instructions without requiring modifications to the internal data path of the core itself [23].

The PCPI wrapper module (pcpi_mac) provides the MAC function as a PCPI interface by monitoring the instruction stream to find the opcode associated with the MAC function. When a valid MAC instruction is detected in the incoming instruction stream, the PCPI wrapper asserts "pcpi_valid" via the PCPI interface and passes the two source operands (pcpi_rs1 and pcpi_rs2) to the MAC logic.

The result produced by the MAC is a 32-bit value which is sent back to the processor through the pcpi_rd signal and has an associated pcpi_ready signal indicating that the MAC has completed execution.

A simple on-chip memory model has also been incorporated into the top-level design of the system to store instructions to help with simulation-based verification [23]. The memory is pre-loaded with MAC instructions that will test the ability of the MAC to correctly detect and execute opcode instructions. This provides a way to closely couple the processor and the custom co-processor at the RTL level.

Therefore, identifying that an instruction executed correctly, integrating the MAC Unit correctly into the processor, and functioning normally in conjunction with ALU operations all would have been done through waveform analysis with a substantial amount of information derived from the performance of this test setup [12].

VI. RESULT

1. Functional Verification of MAC Operation

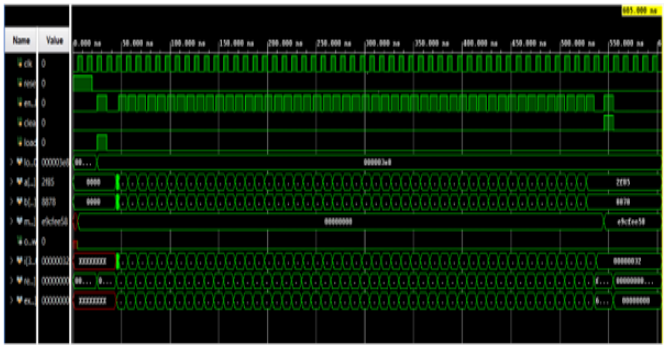


Fig 4. Output of Mac Block

In Figure 4, the simulated waveform depicts the expected multiply-accumulate (MAC) functionality under various control settings. As shown for this specific test case: (1) the operand input values are supplied to the MAC inputs, and (2) the results of successive multiplication cycles are continuously being added to the product output, therefore the final MAC output result matches that of the expected arithmetic value confirmed that the multiplier, accumulator, and control logic function correctly; and (3) The MAC output also demonstrated write-back stability indicating proper forwarding of MAC result through the write-back path of processor to destination registers [23].

2. MAC vs Non-MAC Execution

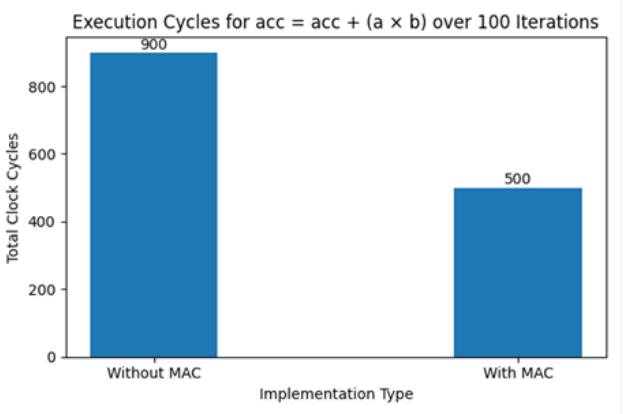


Fig 5. MAC vs Non-MAC Execution

The workload of DSP acc = acc + (a x b) for 100 iterations was tested under identical conditions using both the base PicoRV32 and enhanced PicoRV32 with MACs. The Base core needed 200 instructions and 900 cycles of clock time to complete the workload through standard RISC-V multiply and add instructions. The Enhanced core completed the same workload in 100 instructions and 500 cycles of clock time by using the Proposed Custom MAC instruction. The Enhanced processor reduced both the number of instructions needed for execution by 50% and the number of cycles of clock time by 44% when compared to the Base processor. These results confirm that multiply– accumulate workloads will see an increase in execution efficiency [11].

3. PicoRV32 Validation

In order to confirm that the overall architecture of the PicoRV32 processor, including the custom MAC coprocessor, is functioning correctly and meeting specification requirements, functional simulation and equivalence checking of the hardware were performed to evaluate the correctness, execution delay, and resource utilization. The behavioral simulation of the chip was conducted on the Vivado XSim simulator, in which the instruction memory and testbench environments were set up so that the PCPI interface would trigger the execution of the custom MAC instruction. The simulation waveforms indicate that the debug_pcpi_valid and debug_pcpi_ready signals were properly asserted, which indicates that the processor has successfully detected and offloaded the custom instruction to the MAC co-processor, and has executed it correctly using a valid handshake process.

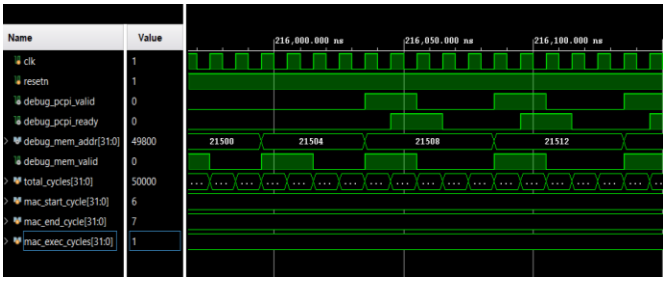


Fig. 6. Simulation waveform showing PCPI handshake during execution of the custom MAC instruction, where the debug_pcpi_valid and debug_pcpi_ready confirms successful single-cycle execution of the MAC coprocessor.

Cycle measurement test bench was used to quantify and analyze cycle-level performance of the chip through dedicated testing methods. Testing shown through cycle-level results of the MAC instruction begin action at cycle 6 with completion of actions completed at cycle 7 showing that the MAC instructions execute in a single cycle for the custom MAC operation. These results support the statement that the proposed custom instruction significantly reduces the execution overhead of execution compared with the traditional RISC-V design as RISC-V systems require multiple sequential instructions (one for multiply then another for add) to accomplish MAC functions.

Resource Type	Utilization	Available	Utilization (%)
LUTs	1338	134600	0.99%
Flip-Flops (FF)	953	269200	0.35%
BRAM	0.5	365	0.14%
DSP Slices	2	740	0.27%
IO	68	400	17.00%

Fig. 7. Synthesis results showing logic and DSP resource utilization.

We also used Xilinx Vivado for synthesis to determine how much additional hardware the proposed design would cost, and the synthesis report indicates that the total resources used for our architecture would be approximately 1338 LUTs, 953 flip-flops and 2 DSPs. There is very little use of DSPs or logic resources to implement the PCPI co-processor for MAC, therefore as designed, the lightweight characteristics of the PicoRV32 architecture are not altered.

The results of the synthesis report, cycle count measurements, and waveform analyses of our implementation confirm the MAC instruction has achieved the desired functional behavior, reduced execution time, and achieved adequate resource utilization. Our results have shown that the use of custom instructions based on PCPI can speed up computations that are dominant in MAC for embedded systems and RISC-V environments.

VII. CONCLUSION

This article describes the creation of a custom multiply-accumulate (MAC) instruction that has been incorporated into an open-source RISC-V processor using the Pico Co-Processor Interface (PCPI). This new architecture allows for MAC operations to be completed faster, while preserving the base datapath of the processor and as a result greatly improving the efficiency of the processor and providing lightweight and modular implementations for processors [1].

The RTL MAC co-processor unit was created and simulated in Verilog and was verified by functional simulations and through synthesis. The functional simulation of the MAC instruction proved that it executes correctly at a single cycle latency, as shown through waveform analysis and cycle-level measurements [5].

The synthesis of the MAC instruction on an FPGA showed very little overhead in LUTs, flip-flops and DSPs and therefore is a suitable design for resource constrained embedded systems. The introduction of the PCPI MAC unit reduces the cost and time required to execute the MAC instruction when compared to traditional Mult instruction implementations of a MAC in a RISC-V processor.

This paper provides an explanation of how to leverage unique ISA extensions to improve computation performance, all while avoiding complexity in the base architecture. Future activities will extend the current architecture by developing custom accelerators that support vector MAC instructions and developing other types of custom accelerators for future use in machine learning or signal processing applications [23].

REFERENCES

[1] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS- 2014-146, Aug. 2014.

[2] K. Y. Kyaw, W. L. Goh, and K. S. Yeo, ''Low-power high-speed multiplier for error-tolerant application,'' in Proc. Int. Conf. Electron Devices Solid State Circuits, 2010, pp. 1–4.

[3] S. Park, D. Kang, J. Kang, and D. Kwon, ''Bratter: An instruction set extension for forward control-flow integrity in RISC-V,'' Sensors, vol. 22, no. 4, p. 1392, Feb. 2022.

[4] H. B. Amor, C. Bernier, and Z. Přikryl, "A RISC-V ISA extension for ultra-low power IoT wireless signal processing," IEEE Transactions on Computers, vol. 71, no. 4, pp. 766–778, Apr. 2022.

[5] M. Cococcioni, F. Rossi, E. Ruffaldi, and S. Saponara, "A lightweight posit processing unit for RISC-V processors in deep neural network applications," IEEE Transactions on Emerging Topics in Computing, vol. 10, no. 4, pp. 1898–1908, Oct. 2022.

[6] M. S. Kumar, D. A. Kumar, and P. Samundiswary, "Design and performance analysis of multiply-accumulate (MAC) unit," in Proc. Int. Conf. Circuits, Power and Computing Technologies, 2014, pp. 1084–1089.

[7] S. Dutt, A. Chauhan, R. Bhadoriya, S. Nandi, and G. Trivedi, "A high-performance energy-efficient hybrid redundant MAC for error-resilient applications," in Proc. Int. Conf. VLSI Design, 2015, pp. 351–356.

[8] A. Cilardo, D. De Caro, N. Petra, F. Caserta, N. Mazzocca, E. Napoli, and A. G. M. Strollo, "High-speed speculative multipliers based on speculative carry-save tree," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 61, no. 12, pp. 3426–3435, Dec. 2014.

[9] I. Qiqieh, R. Shafik, G. Tarawneh, D. Sokolov, and A. Yakovlev, "Energy-efficient approximate multiplier design using bit significance-driven logic compression," in Proc. Design, Automation & Test in Europe Conf., 2017, pp. 7–12.

[10] N. Petra, D. De Caro, V. Garofalo, E. Napoli, and A. G. M. Strollo, "Design of fixed-width multipliers with linear compensation function," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 58, no. 5, pp. 947–960, May 2011.

[11] M. de la Guia Solaz, W. Han, and R. Conway, "A flexible low-power DSP with a programmable truncated multiplier," IEEE Transactions on Circuits and Systems, vol. 59, no. 11, pp. 2555– 2568, Nov. 2012.

[12] J. Ball, "Designing soft-core processors for FPGAs," in Processor Design: System-on-Chip Computing for ASICs and FPGAs, J. Nurmi, Ed. Springer, 2007, pp. 229–256.

[13] T. Chen et al., "A survey on neural network accelerators," ACM Computing Surveys, vol. 53, no. 1, pp. 1–29, 2020.

[14] J. McAllister, "FPGA-based DSP," in Handbook of Signal Processing Systems, S. S. Bhattacharyya et al., Eds. Springer, 2010, pp. 363–392.

[15] M. Gautschi et al., "Near-threshold RISC-V core with DSP extensions for scalable IoT sensing," IEEE Transactions on VLSI Systems, vol. 25, no. 10, pp. 2700– 2713, Oct. 2017, Doi: 10.1109/TVLSI.2017.2654506.

[16] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V instruction set manual, volume I: User-level ISA, version 2.0," Univ. of California, Berkeley, Tech. Rep. UCB/EECS-2014-54, 2014.

[17] Y. Lee *et al.*, "A 45nm 1.3GHz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators," *IEEE J. Solid-State Circuits*, vol. 50, no. 1, pp. 287–296, Jan. 2015.

[18] F. Zaruba and L. Benini, "The cost of application-class processing: Energy and performance analysis of a Linux- ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2629–2640, Nov. 2019.

[19] F. Schuiki, F. Zaruba, T. Hoefler, and L. Benini, "Snitch: A tiny PULP RISC-V core for ultra-low-power systems," in *Proc. Design, Automation & Test in Europe (DATE)*, 2020, pp. 103–108.

[20] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[21] S. K. Mathew *et al.*, "10-nm CMOS design and analysis of a high-performance low-power multiplier," *IEEE J. Solid- State Circuits*, vol. 52, no. 12, pp. 3246–3257, Dec. 2017.

[22] R. G. Dreslinski, M. Wieckowski, D. Blaauw, D. Sylvester, and T. Mudge, "Near-threshold computing: Reclaiming Moore's law through energy efficient integrated circuits," *Proc. IEEE*, vol. 98, no. 2, pp. 253–266, Feb. 2010.

[23] L. Benini, A. Pullini, and D. Rossi, "Ultra-low-power computing platforms for IoT and edge AI," IEEE Micro, vol. 41, no. 2, pp. 8–17, Mar.–Apr. 2021.